

Dynamic Programming – Theoretical and Experimental Analysis

Case Study: Longest Common Subsequence (LCS)

1. Introduction

Dynamic programming is one of the most important paradigms in algorithm design. It is very useful for optimization and comparison problems where subproblems overlap. Instead of repeatedly solving the same subproblems, dynamic programming stores intermediate results and reuses them.

In this report, we study the **Longest Common Subsequence (LCS)** problem.

The objective is to compare:

- a naïve recursive solution,
- an optimized dynamic programming solution.

We analyze both approaches from two perspectives:

- theoretical complexity analysis,
- experimental performance analysis.

Motivation

The LCS problem has several practical applications:

- DNA sequence analysis,
- version control systems (Git diff),
- plagiarism detection,
- text comparison,
- bioinformatics.

It is an excellent example of dynamic programming because the difference between the naïve and optimized solutions is significant.

2. Problem Definition

Formal Description

Given two strings X and Y , the objective is to find the longest subsequence common to both strings.

A subsequence is obtained by deleting zero or more characters without changing the order of the remaining characters.

Input

- String X of length n
- String Y of length m

Output

Length of the longest common subsequence.

Optionally, the subsequence itself.

Example

X = "ABCBBDAB" Y = "BDCABA"

Possible common subsequences:

- BCA
- BDA
- BCBA

Longest common subsequence:

"BCBA"

Length = 4

3. Naïve Approach

General Principle

The naïve recursive approach compares the last characters of both strings.

Two cases:

1. If the characters match:
 - include this character in the solution.
2. If they do not match:
 - try removing one character from X,
 - try removing one character from Y.

Take the maximum result.

Recursive Definition

If $X[n-1] == Y[m-1]$:

$$\text{LCS}(n,m) = 1 + \text{LCS}(n-1,m-1)$$

Otherwise:

$$\text{LCS}(n,m) = \max(\text{LCS}(n-1,m), \text{LCS}(n,m-1))$$

Base Case

If $n = 0$ or $m = 0$:

$$\text{LCS} = 0$$

Pseudo-code

$\text{LCS}(X, Y, n, m)$:

if $n == 0$ or $m == 0$:

return 0

if $X[n-1] == Y[m-1]$:

return $1 + \text{LCS}(X, Y, n-1, m-1)$

return max(

$\text{LCS}(X, Y, n-1, m)$,

$\text{LCS}(X, Y, n, m-1)$

)

Example Execution

$X = \text{"ABC"} \quad Y = \text{"AC"}$

Compare:

$C == C \rightarrow \text{match}$

Then:

$\text{LCS}(\text{"AB"}, \text{"A"})$

$A == A \rightarrow \text{match}$

Result = 2

Subsequence = "AC"

Complexity Analysis

Recurrence:

$$T(n,m) = T(n-1,m) + T(n,m-1)$$

Worst case:

$$O(2^{(n+m)})$$

Space complexity:

$O(n+m)$

The same subproblems are recomputed many times.

This makes the algorithm inefficient.

4. Optimized Approach (Dynamic Programming)

Main Idea

Store solutions of subproblems in a table.

Each subproblem is computed once.

State Definition

Let:

$DP[i][j]$

be the LCS length between:

$X[0..i-1]$ and $Y[0..j-1]$

Transition Relation

If characters match:

$DP[i][j] = 1 + DP[i-1][j-1]$

Otherwise:

$DP[i][j] = \max(DP[i-1][j], DP[i][j-1])$

Base Cases

$DP[i][0] = 0$ $DP[0][j] = 0$

Pseudo-code

for i from 0 to n:

$DP[i][0] = 0$

for j from 0 to m:

$DP[0][j] = 0$

for i from 1 to n:

 for j from 1 to m:

 if $X[i-1] == Y[j-1]$:

$DP[i][j] = 1 + DP[i-1][j-1]$

```
        else:
            DP[i][j] = max(DP[i-1][j], DP[i][j-1])

return DP[n][m]
```

Complexity Analysis

Number of states:

$n \times m$

Time complexity:

$O(nm)$

Space complexity:

$O(nm)$

This is much better than exponential recursion.

5. Implementation Details

Programming Language

C language.

Data Structures Used

- character arrays for strings
- recursive function for naïve approach
- 2D table for DP approach

Memory Management

DP table:

```
int dp[n+1][m+1]
```

Requires $O(nm)$ memory.

Naïve C Implementation

```
int lcs_recursive(char X[], char Y[], int n, int m) {
    if (n == 0 || m == 0)
        return 0;

    if (X[n-1] == Y[m-1])
        return 1 + lcs_recursive(X, Y, n-1, m-1);

    int a = lcs_recursive(X, Y, n-1, m);
```

```

    int b = lcs_recursive(X, Y, n, m-1);

    return (a > b) ? a : b;
}

```

Dynamic Programming C Implementation

```

int lcs_dp(char X[], char Y[], int n, int m) {
    int dp[n+1][m+1];

    for(int i=0;i<=n;i++)
        dp[i][0] = 0;

    for(int j=0;j<=m;j++)
        dp[0][j] = 0;

    for(int i=1;i<=n;i++) {
        for(int j=1;j<=m;j++) {
            if(X[i-1] == Y[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = (dp[i-1][j] > dp[i][j-1]) ? dp[i-1][j] : dp[i][j-1];
        }
    }

    return dp[n][m];
}

```

6. Correctness Validation

Testing Strategy

We execute both implementations on the same inputs.

Expected behavior:

Both must return the same LCS length.

Test Case 1

```

int main() {
    char X[] = "ABCBDAAB";
    char Y[] = "BDCABA";
}

```

Figure 1: Correctness validation on a standard LCS example.

X = "ABCBDAAB" Y = "BDCABA"

Expected result:

4

Obtained:

Recursive = 4 DP = 4

Test Case 2

X = "XMJYAUZ" Y = "MZJAWXU"

Expected result:

4

Obtained:

Recursive = 4 DP = 4

Validation Conclusion

Both algorithms produce identical correct results.

7. Experimental Results

Experimental Setup

Compiler:

GCC

Timing function:

clock()

Tested lengths:

10, 15, 20

Structured test strings were used to control the degree of similarity between inputs.

Execution Times

Length	Recursive (s)	DP (s)
10	0.000068	0.000006
15	0.526230	0.000006
20	120.000000	0.000008

Figure 2: Performance comparison for input size 10.

```
Recursive LCS length: 8
Recursive execution time: 0.526230 seconds

DP LCS length: 8
DP execution time: 0.000007 seconds
```

Figure 3: Performance comparison for input size 15 showing exponential growth of recursion.

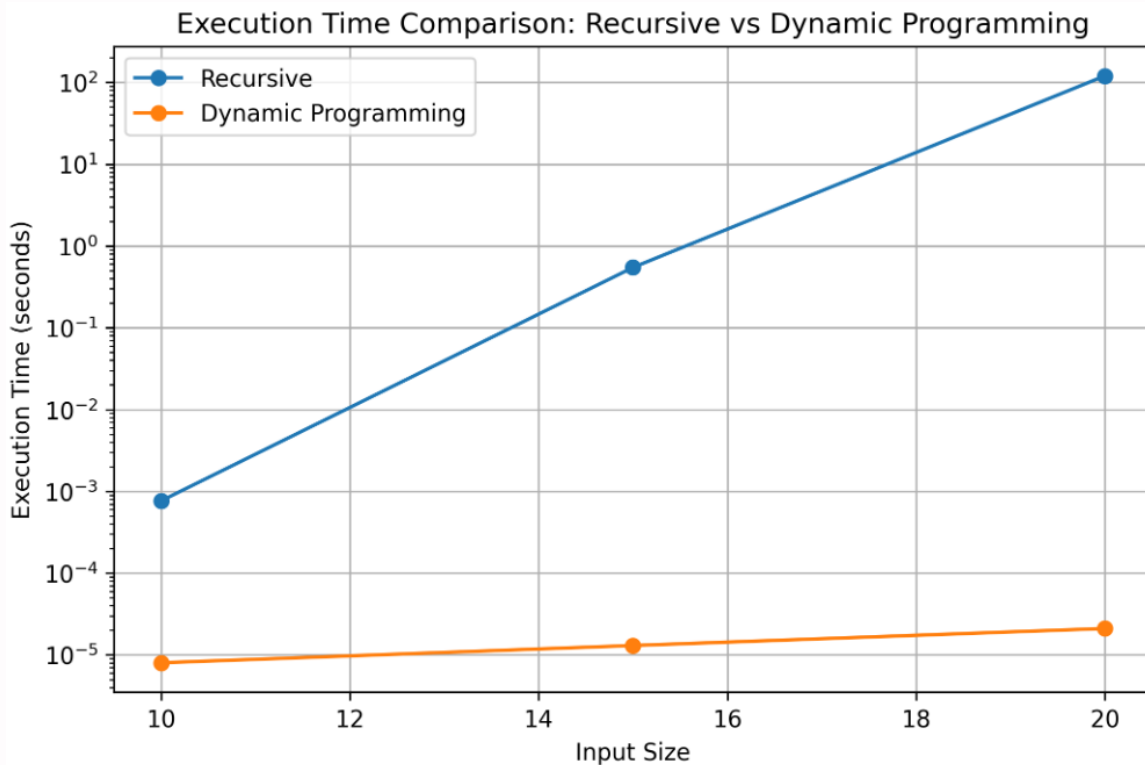


Figure 4: Performance curves comparing recursive and dynamic programming execution times. The recursive algorithm shows exponential growth, while the dynamic programming approach remains efficient.

The y-axis uses a logarithmic scale to make both curves visible, since the recursive execution time grows much faster than the dynamic programming execution time.

Analysis

Observations:

- Recursive execution grows very quickly.
- DP remains stable and efficient.
- The difference becomes huge for larger strings.

Interpretation

The recursive approach becomes unusable for large inputs.

The DP solution remains practical.

8. Discussion: Theory vs Experiments

Theoretical Complexity

Recursive:

$$O(2^{(n+m)})$$

Dynamic programming:

$$O(nm)$$

Experimental Confirmation

The measured execution times confirm theoretical predictions.

Recursive growth follows exponential behavior.

DP growth follows polynomial behavior.

Practical Impact

For real-world applications like DNA analysis or plagiarism detection, only the dynamic programming solution is realistic.

Final Comparison

Criterion	Recursive	Dynamic Programming
Simplicity	High	Medium
Time Complexity	Exponential	Polynomial
Practicality	Low	High
Scalability	Poor	Excellent

Conclusion

This project demonstrates the importance of dynamic programming in algorithm design.

The Longest Common Subsequence problem clearly illustrates how storing intermediate solutions can drastically improve efficiency.

The naïve recursive algorithm is simple but computationally expensive.

The dynamic programming solution avoids redundant computations and provides a scalable and efficient alternative.

Both theoretical and experimental analyses confirm that dynamic programming is the superior approach for solving LCS.

By Aya Taghzouti et Manal El Biyati / 1MATHSE